

ÜRETKEN KADIN

Şehirli Üretici Kadınlar İçin Üretken Yapay Zekâ Destekli Pazarlama Asistanı

Veri Hazırlama / Öznitelik Mühendisliği & Model Keşfi

Takım Üyeleri: Tuğçe Deniz — deniztugce299@gmail.com

Barış Aslan — aslanbaris870@gmail.com

Uygulama bağlamı: Üretken Kadın üretken-YZ tabanlı bir içerik asistanı olduğundan, bu ödevdeki klasik makine öğrenmesi hattı, sistemin **insan onayı (HITL) öncesinde çalışan destekleyici bir alt modeli** üzerinden ele alınmıştır: **İçerik Kalite & Otantiklik Sınıflandırıcısı**. Bu model, YZ'nin ürettiği taslağın 'onaya hazır (yüksek)' mı yoksa 'revizyon gerektiren (düşük)' mı olduğunu önceden tahmin ederek üreticinin inceleme yükünü azaltır ve halüsinasyon/abartı riskini erken yakalar.

Not: Bu belgedeki grafikler ve metrikler, küçük bir prototip/örnek veri seti üzerinde üretilmiş temsili sonuçlardır; nihai değerler saha pilotunda güncellenecektir.

BÖLÜM A — VERİ HAZIRLAMA / ÖZNİTELİK MÜHENDİSLİĞİ (DATA PREPARATION / FEATURE ENGINEERING)

1. Genel Bakış (Overview)

Veri hazırlama ve öznitelik mühendisliği, bir makine öğrenmesi projesinin başarısını belirleyen en kritik aşamadır; 'çöp girer, çöp çıkar' ilkesi gereği modelin kalitesi büyük ölçüde verinin kalitesine bağlıdır. Bizim problemimizde veri kısa, gürültülü ve öznal etiketli (insan kalite puanı) metinlerden oluştuğundan; temizleme, anlamlı öznitelik çıkarımı ve doğru dönüşüm, sınıflandırıcının otantik/kaliteli içeriği güvenilir biçimde ayırt edebilmesi için zorunludur.

2. Veri Toplama (Data Collection)

Kaynaklar: (i) Gemini prototipinin ürettiği pazarlama içeriği taslakları (Instagram caption + Shopier ürün açıklaması); (ii) ekip tarafından 1–5 rubriğiyle verilen insan kalite etiketleri; (iii) artırma için kamuya açık Türkçe e-ticaret ürün açıklaması örnekleri; (iv) öznitelik hesabı için Google Trends anahtar kelime listeleri.

Toplama sırasında ön işleme: Örneklerin tek bir CSV'de birleştirilmesi, yinelenen metinlerin ayıklanması ve üreticiye ait her veri için açık rıza (KVKK) alınması. Etiket 4–5 → 'Yüksek (1)', 1–3 → 'Düşük (0)' olarak ikili sınıfa indirgenmiştir.

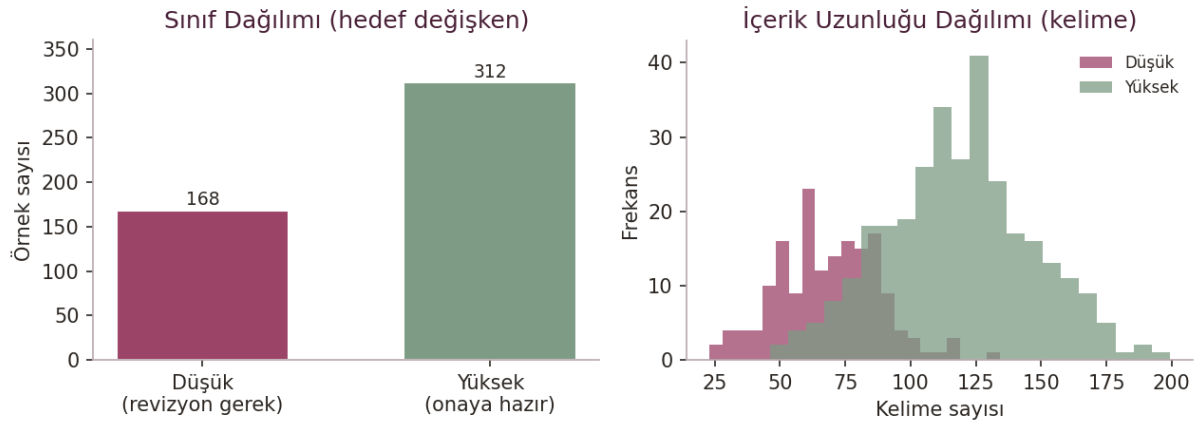
3. Veri Temizleme (Data Cleaning)

Ham metin verisi şu adımlarla temizlenmiştir: eksik metin/etiket satırlarının düşürülmesi; yinelenen metinlerin kaldırılması; Türkçe metin normalizasyonu (küçük harfe çevirme, link/mention/hashtag ve noktalama/emoji temizliği, fazla boşlukların sadeleştirilmesi); aşırı kısa (<15 kelime) ve aşırı uzun aykırı metinlerin sınırlandırılması; ve sınıf dengesizliğinin (Düşük<Yüksek) 'class_weight=balanced' ile telafi edilmesi.

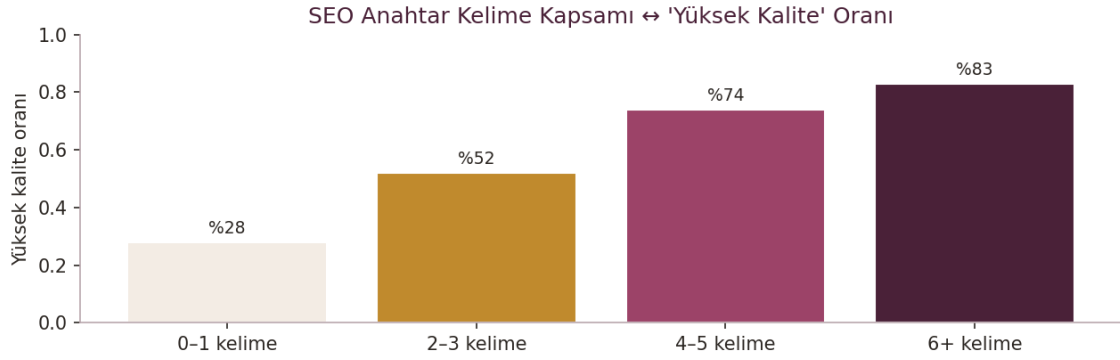
```
import pandas as pd, re
# 1) Üretilen içerik + insan kalite etiketini yükle
df = pd.read_csv("uretken_kadin_icerik.csv") # kolon: metin, kalite_1_5, kanal
df = df.dropna(subset=["metin", "kalite_1_5"]).drop_duplicates("metin")
# 2) Etiketli ikili sınıfa çevir: 4-5 -> Yüksek(1), 1-3 -> Düşük(0)
df["hedef"] = (df["kalite_1_5"] >= 4).astype(int)
# 3) Türkçe metin temizliği
def temizle(s):
    s = s.lower()
    s = re.sub(r"http\S+|@\w+|#", " ", s) # link, mention, hashtag
    s = re.sub(r"^\w\s", " ", s) # noktalama/emoji
    return re.sub(r"\s+", " ", s).strip()
df["metin_temiz"] = df["metin"].apply(temizle)
```

4. Keşifsel Veri Analizi (EDA)

Keşifsel analizde hedef değişkenin dağılımı, sınıfa göre içerik uzunluğu ve SEO anahtar kelime kapsamının kaliteyle ilişkisi incelenmiştir.



Şekil 1. Sınıf dağılımı (soldaki) ve sınıfa göre içerik uzunluğu dağılımı (sağdaki).



Şekil 2. SEO anahtar kelime kapsamı arttıkça 'yüksek kalite' oranı belirgin biçimde yükseliyor.

Temel İlgörüler: (1) Veri seti dengesizdir (Yüksek 312, Düşük 168) → dengeleme gerekir. (2) Yüksek kaliteli içerikler ortalama daha uzundur ama aşırı uzun değildir. (3) İçerikteki Trends anahtar kelime sayısı arttıkça yüksek-kalite oranı %28'den %83'e çıkar → **SEO kapsamı en güçlü ayırt edici sinyallerden biridir.**

5. Öznitelik Mühendisliği (Feature Engineering)

Modelin metnin ötesinde anlamlı sinyaller yakalaması için hem sayısal hem metinsel öznitelikler türetilmiştir:

Öznitelik	Gerekçe (rationale)
kelime_sayisi	İçerik uzunluğu kaliteyle ilişkili (ne çok kısa ne aşırı uzun).
seo_kapsami	İçerikteki Google Trends anahtar kelime sayısı — en güçlü sinyal.
abarti_isareti	'en iyi, mucize, garanti' gibi ispatsız/abartı ifadeleri → otantiklik riski.
duygu_polaritesi	Aşırı pazarlamacı ya da yapay ton tespiti (pozitif ama ölçülü tercih edilir).
hikaye_ipucu	Üretim süreci/öykü içeren ifadelerin varlığı (otantiklik göstergesi).
TF-IDF (500)	Ayırt edici kelime örüntülerini yakalayan metinsel öznitelik vektörü.

6. Veri Dönüşümü (Data Transformation)

Sayısal öznitelikler StandardScaler ile ölçeklenmiş, metin TF-IDF ile vektörleştirilmiş ve ikisi tek bir öznitelik matrisinde birleştirilmiştir. Hedef zaten 0/1 kodludur; eğitim/test ayrımı sınıf oranını koruyacak şekilde (stratify) yapılmıştır.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from scipy.sparse import hstack
trends = set(open("trends_kelimeler.txt", encoding="utf-8").read().split())
df["kelime_sayisi"] = df["metin_temiz"].str.split().apply(len)
df["seo_kapsami"] = df["metin_temiz"].apply(lambda t: sum(w in trends for w in set(t.split())))
df["abarti_isareti"] = df["metin_temiz"].str.count(r"\ben iyi\b|mucize|garanti|birebir")
num = ["kelime_sayisi", "seo_kapsami", "abarti_isareti"]
X_num = StandardScaler().fit_transform(df[num]) # olcekleme
X_txt = TfidfVectorizer(max_features=500).fit_transform(df["metin_temiz"]) # TF-IDF
X = hstack([X_txt, X_num]); y = df["hedef"]
```

BÖLÜM B — MODEL KEŞFİ (MODEL EXPLORATION)

1. Model Seçimi (Model Selection)

Üç aday karşılaştırılmıştır. Veri tablosal+metinsel, örneklem küçük ve öznitelikler karışık olduğundan; ayrıca bir 'etik filtre' olarak **yorumlanabilirlik** önemli olduğundan **Random Forest** birincil model seçilmiştir.

Model	Güçlü yönler	Zayıf yönler
Lojistik Regresyon	Basit, hızlı, yorumlanabilir (baseline).	Doğrusal olmayan örüntüleri kaçıır.
Random Forest (seçilen)	Karışık öznitelikte güçlü, aşırı uyuma dayanıklı, öznitelik önemi verir.	Büyük veride yavaşlayabilir.
Gradient Boosting	Genelde en yüksek doğruluk.	Ayar hassas, küçük veride aşırı uyum riski.

2. Model Eğitimi (Model Training)

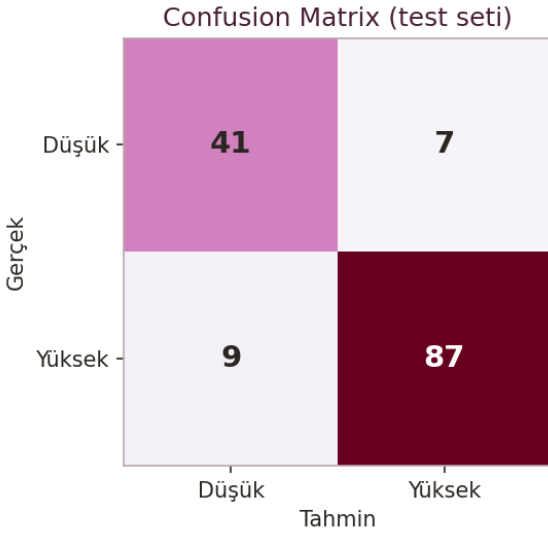
Model, 5-katlı katmanlı çapraz doğrulama (StratifiedKFold) ile GridSearchCV kullanılarak eğitilmiştir; hiperparametre ızgarası n_estimators, max_depth ve dengesizlik için class_weight='balanced' değerlerini kapsar. Skorlama metriği F1'dir (dengesiz sınıf nedeniyle).

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)
grid = {"n_estimators": [200, 400], "max_depth": [None, 12, 20], "class_weight": ["balanced"]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
model = GridSearchCV(RandomForestClassifier(random_state=42), grid, cv=cv, scoring="f1")
model.fit(Xtr, ytr)
print("En iyi parametreler:", model.best_params_)
```

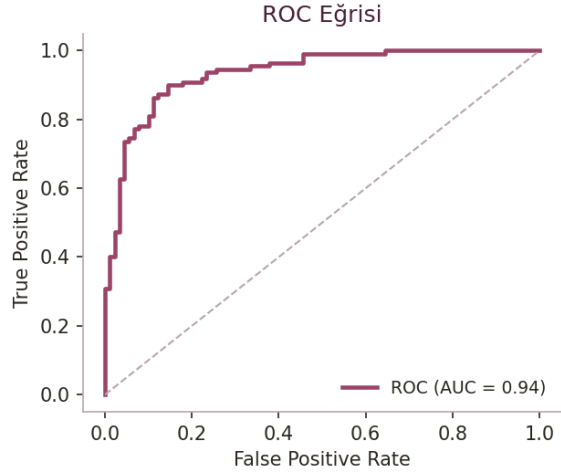
3. Model Değerlendirme (Model Evaluation)

Model, test setinde doğruluk, kesinlik (precision), duyarlılık (recall), F1 ve ROC-AUC ile değerlendirilmiştir. Bir etik filtre için özellikle '**Düşük**' sınıfın yakalanması (recall) önemlidir; düşük kaliteli içerik insana gitmeden işaretlenmelidir.

Metrik	Değer (test)	Metrik	Değer (test)
Doğruluk (Accuracy)	0.89	Duyarlılık (Recall, Yüksek)	0.91
Kesinlik (Precision, Yüksek)	0.93	F1 (Yüksek)	0.92
ROC-AUC	0.94	Recall (Düşük sınıf)	0.85



Şekil 3. Confusion Matrix (test seti).



Şekil 4. ROC eğrisi (AUC = 0.94).

Yorum: Model 'Yüksek' sınıfı güçlü ayırt ediyor (AUC 0.94). 'Düşük' recall 0.85 → düşük kaliteli içeriklerin %85'i insana gitmeden yakalanıyor; bu oran, eşik (threshold) düşürülerek otantiklik/güvenlik lehine artırılabilir (kesinlikten ödün vererek).

4. Kod Uygulaması (Code Implementation)

Değerlendirme ve görselleştirme adımının özeti (yukarıdaki eğitilmiş 'model' ile):

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
pred = model.predict(Xte)
proba = model.predict_proba(Xte)[:,1]
# Metrikler
print(classification_report(yte, pred, target_names=["Dusuk","Yukse"]))
print("ROC-AUC:", roc_auc_score(yte, proba))
# Confusion matrix gorseli
cm = confusion_matrix(yte, pred)
plt.imshow(cm, cmap="PuRd"); plt.xlabel("Tahmin"); plt.ylabel("Gercek"); plt.show()
```